

LAPORAN TUGAS BESAR ROBOCODE STRATEGI ALGORITMA



Disusun oleh:

- 1. Mandanta Guru Singa (124140147)**
- 2. Rachman Hady (124140129)**
- 3. Gathan Mahendra (124140201)**

Teknik Informatika

STRATEGI ALGORITMA RD (IF2211)

Winda Yulita, M.Cs.

RD

INSTITUT TEKNOLOGI SUMATERA LAMPUNG SELATAN 2026





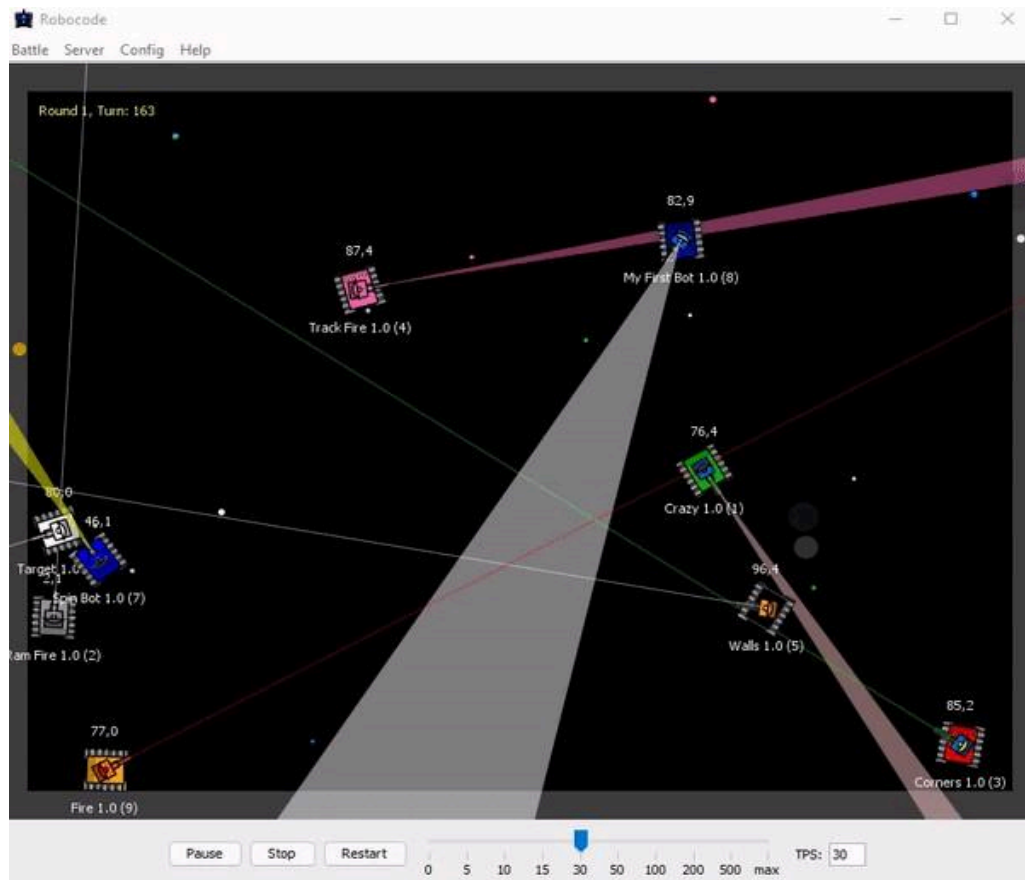


DAFTAR ISI

JUDUL.....	1
DAFTAR ISI.....	5
DESKRIPSI TUGAS.....	6
1. Rounds dan Turns.....	8
2. Batas Waktu Giliran.....	9
3. Energi.....	10
4. Peluru.....	10
5. Panas Meriam (Gun Heat).....	10
6. Tabrakan.....	10
7. Bagian Tubuh Tank.....	11
8. Pergerakan.....	11
9. Berbelok.....	11
10. Pemindaian.....	12
11. Skor.....	12
LANDASAN TEORI.....	14
2.1 Dasar Teori Algoritma Greedy.....	14
2.1.1 Filosofi dan Konsep Dasar.....	14
2.1.2 Elemen-Elemen Pembentuk Algoritma Greedy.....	14
2.1.3 Analisis Kompleksitas dan Limitasi.....	15
2.2 Cara Kerja Program secara Umum.....	15
2.2.1 Paradigma Event-Driven pada Robocode Tank Royale.....	15
2.2.2 Diskritisasi Waktu (Rounds, Turns, dan Timeout).....	15
2.2.3 Anatomi dan Kinematika Agen.....	15
2.2.4 Fisika Permainan dan Manajemen Sumber Daya.....	16
APLIKASI STRATEGI GREEDY.....	17
3.1 Mapping Persoalan Robocode Menjadi Elemen Greedy.....	17
3.2 Eksplorasi 4 Alternatif Solusi Greedy.....	18
3.3 Analisis Efisiensi dan Efektivitas.....	18
3.3.1 Efisiensi Komputasi.....	18
3.3.2 Efektivitas Strategi.....	19
3.4 Strategi Greedy yang Dipilih.....	20
IMPLEMENTASI ALGORITMA GREEDY.....	21
4.1 Implementasi 4 Alternatif Solusi (Pseudocode).....	21
4.1.1 Alternatif Solusi 1: Algoritma Adaptive Predictive dan Moving Average Filter (MainBot)	21
4.1.2 Alternatif Solusi 2: Algoritma Defensive Sniper (GreedySniper).....	22
4.1.3 Alternatif Solusi 3: Algoritma Aggressive Rammer (GreedyRammer).....	23
4.1.4 Alternatif Solusi 4: Algoritma Evasive Dodger (GreedyDodger).....	23
4.2 Struktur Data, Fungsi, dan Prosedur.....	24
4.2.1. Struktur Kelas Data dan Atribut State.....	24
4.2.2. Variabel Global (State Tracking dari API).....	24
4.2.3. Fungsi dan Prosedur Utama (Method).....	24
4.3 Pengujian Pertandingan Free-for-All (FFA) 4 Bot.....	25
4.4 Analisis Hasil Pengujian.....	26
4.4.1 Keberhasilan Strategi Greedy.....	26
4.4.2 Kondisi Strategi Greedy Tidak Optimal.....	26
KESIMPULAN DAN SARAN.....	28

5.1 Kesimpulan.....	28
5.2 Saran.....	28
LAMPIRAN.....	29
DAFTAR PUSTAKA.....	30

DESKRIPSI TUGAS



Gambar 1. Robocode Tank Royale

Robocode adalah permainan pemrograman yang bertujuan untuk membuat kode bot dalam bentuk tank virtual untuk berkompetisi melawan bot lain di arena. Pertempuran Robocode berlangsung hingga bot-bot bertarung hanya tersisa satu seperti permainan Battle Royale, karena itulah permainan ini dinamakan Tank Royale. Nama Robocode adalah singkatan dari "Robot code," yang berasal dari [versi asli/pertama permainan ini](#). Robocode Tank Royale adalah evolusi/versi berikutnya dari permainan ini, di mana bot dapat berpartisipasi melalui Internet/jaringan. Dalam permainan ini, pemain berperan sebagai programmer bot dan tidak memiliki kendali langsung atas permainan. Pemain hanya bertugas untuk membuat program yang menentukan logika atau "otak" bot. Program yang dibuat akan berisi instruksi tentang cara bot bergerak, mendeteksi bot lawan, menembakkan senjatanya, serta bagaimana bot bereaksi terhadap berbagai kejadian selama pertempuran.

Pada Tugas Besar pertama Strategi Algoritma ini, mahasiswa diminta untuk membuat sebuah bot yang nantinya akan dipertandingkan satu sama lain. Tentunya mahasiswa harus menggunakan **strategi greedy** dalam membuat bot ini.

Komponen-komponen dari permainan ini antara lain:

1. Rounds dan Turns

Pertempuran dapat terdiri dari beberapa rounds. Secara default, satu pertempuran berisi 10 rounds, di mana setiap rounds akan memiliki pemenang dan yang kalah.

Setiap round dibagi menjadi beberapa turns, yang merupakan unit waktu terkecil. Satu turn adalah satu ketukan waktu dan satu putaran permainan. Jumlah turn dalam satu round tergantung pada berapa lama waktu yang dibutuhkan hingga hanya tersisa bot terakhir yang bertahan.

Pada setiap turn, sebuah bot dapat:

- Menggerakkan bot, memindai musuh, dan menembakkan senjata.
- Bereaksi terhadap peristiwa seperti saat bot terkena peluru atau bertabrakan dengan bot lain atau dinding.
- Perintah untuk bergerak, berputar, memindai, menembak, dan sebagainya dikirim ke server untuk setiap turn.

Perlu diperhatikan bahwa [API \(Application Programming Interface\)](#) bot resmi secara otomatis mengirimkan niat bot ke server di balik layar, sehingga Anda tidak perlu mengkhawatirkannya, kecuali jika Anda membuat API Bot sendiri.

Pada setiap turn, bot akan secara otomatis menerima informasi terbaru tentang posisinya dan orientasinya di medan perang. Bot juga akan mendapatkan informasi tentang bot musuh ketika mereka terdeteksi oleh pemindai.

Perlu diketahui bahwa game engine yang akan digunakan pada tugas besar ini tidak mengikuti aturan default mengenai komponen Round & Turns.

2. Batas Waktu Giliran

Penting untuk dicatat bahwa setiap bot memiliki batas waktu untuk setiap turn yang disebut turn timeout, biasanya antara 30-50 ms (dapat diatur sebagai aturan pertempuran). Ini berarti bahwa bot tidak bisa mengambil waktu sebanyak yang mereka inginkan untuk bergerak dan menyelesaikan turn saat ini.

Setiap kali turn baru dimulai, penghitung waktu ulang diatur ulang dan mulai berjalan. Jika batas waktu tercapai dan bot tidak mengirimkan pergerakannya untuk turn tersebut, maka tidak ada perintah yang dikirim ke server. Akibatnya, bot akan melewatkan turn tersebut. Jika bot melewatkan turn, ia tidak akan bisa menyesuaikan gerakannya atau menembakkan senjatanya karena server tidak menerima perintah tepat waktu sebelum turn berikutnya dimulai.

3. Energi

Semua bot memulai permainan dengan jumlah energi awal sebanyak 100 poin energi.

- Bot akan kehilangan energi jika ditembak atau ditabrak oleh bot musuh.
- Bot juga akan kehilangan energi jika menembakkan meriamnya.
- Bot akan mendapatkan energi jika peluru dari meriamnya mengenai musuh. Energi yang didapat akan lebih banyak 3 kali lipat dari energi yang digunakan untuk menembakkan peluru.
- Bot dengan energi nol akan dinonaktifkan dan tidak bisa bergerak. Jika bot terkena serangan dalam keadaan ini, bot akan hancur.

4. Peluru

Semakin banyak energi (daya tembak) yang digunakan untuk menembakkan peluru, semakin berat peluru tersebut dan semakin lambat gerakannya. Namun, peluru yang lebih berat juga menghasilkan lebih banyak kerusakan dan memungkinkan bot mendapatkan lebih banyak energi saat mengenai bot musuh.

Seperti disebutkan sebelumnya, peluru yang lebih berat akan bergerak lebih lambat. Ini berarti akan membutuhkan waktu lebih lama untuk mencapai target, meningkatkan risiko peluru tidak mengenai sasaran. Sebaliknya, peluru yang lebih ringan bergerak lebih cepat, sehingga lebih mudah mengenai target, tetapi peluru ringan tidak memberikan banyak poin energi saat mengenai bot musuh.

5. Panas Meriam (Gun Heat)

Saat menembakkan peluru, meriam akan menjadi panas. Peluru yang lebih berat menghasilkan lebih banyak panas dibandingkan peluru yang lebih ringan. Ketika meriam terlalu panas, bot tidak dapat menembak hingga suhu meriam turun ke nol. Selain itu, meriam juga sudah dalam keadaan panas di awal round dan perlu waktu untuk mendingin sebelum bisa digunakan untuk pertama kalinya.

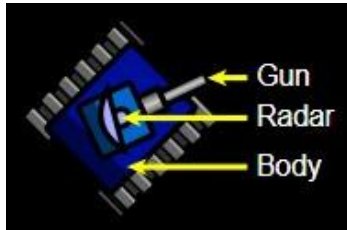
6. Tabrakan

Perlu diperhatikan bahwa bot akan menerima kerusakan jika menabrak dinding (batas arena), yang disebut wall damage. Hal yang sama juga terjadi jika bot bertabrakan dengan bot lain.

Jika bot menabrak bot musuh dengan bergerak maju, ini disebut ramming (menabrak dengan sengaja), yang akan memberikan sedikit skor tambahan bagi bot yang menyerang.

7. Bagian Tubuh Tank

Tubuh tank terdiri dari 3 bagian:



Body adalah bagian utama dari tank yang digunakan untuk menggerakkan tank.

Gun digunakan untuk menembakkan peluru dan dapat berputar bersama *body* atau independen dari *body*.

Radar digunakan untuk memindai posisi musuh dan dapat berputar bersama *body* atau independen dari *body*.

8. Pergerakan

Bot dapat bergerak maju dan mundur hingga kecepatan maksimum. Dibutuhkan beberapa giliran untuk mencapai kecepatan maksimum. Bot dapat mengalami percepatan maksimum sebesar 1 unit per giliran dan pengereman dengan perlambatan maksimum 2 unit per giliran. Percepatan dan perlambatan maksimum tidak bergantung pada kecepatan bot saat itu.

9. Berbelok

Seperti yang disebutkan sebelumnya, bagian tubuh, turret (meriam), dan radar dapat berputar secara independen satu sama lain. Jika turret atau radar tidak diputar, maka keduanya akan mengarah ke arah yang sama dengan tubuh bot.

Setiap bagian tubuh memiliki kecepatan putar yang berbeda. Radar adalah bagian tercepat dan dapat berputar hingga 45 derajat per giliran, yang berarti dapat berputar 360 derajat dalam 8 giliran. Turret dan meriam dapat berputar hingga 20 derajat per giliran.

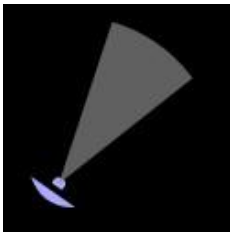
Bagian paling lambat adalah tubuh tank, yang dalam kondisi terbaik dapat berputar hingga 10 derajat per giliran. Namun, ini bergantung pada kecepatan bot saat ini. Semakin cepat bot bergerak, semakin lambat kemampuannya untuk berbelok.

Perlu diperhatikan bahwa tidak ada energi yang dikonsumsi saat bot bergerak atau berbelok.

10. Pemindaian

Aspek penting dalam Robocode adalah memindai bot musuh menggunakan radar. Radar dapat mendeteksi bot dalam jangkauan hingga 1200 piksel. Musuh yang berada lebih dari 1200 piksel dari bot tidak dapat terdeteksi atau dipindai oleh radar.

Penting untuk diperhatikan bahwa sebuah bot hanya dapat memindai bot musuh yang berada dalam jangkauan sudut pemindaian (scan arc)-nya. Sudut pemindaian ini merupakan "sapuan radar" dari arah radar sebelumnya ke arah radar saat ini dalam satu giliran.



Jika radar tidak bergerak dalam suatu giliran, artinya radar tetap mengarah ke arah yang sama seperti pada giliran sebelumnya, maka sudut pemindaian akan menjadi nol derajat, dan bot tidak akan dapat mendeteksi musuh.



Oleh karena itu, sangat disarankan untuk selalu mengubah arah radar agar tetap dapat memindai musuh.

11. Skor

Pada akhir pertempuran, setiap bot akan diranking berdasarkan total skor yang diperoleh masing-masing bot selama keseluruhan pertempuran. Tentunya, tujuan utama pada tugas besar ini adalah membuat bot yang memberikan skor setinggi mungkin. Berikut adalah rincian komponen skor pada pertempuran:

- **Bullet Damage:** Bot mendapatkan **poin sebesar *damage*** yang dibuat kepada bot musuh menggunakan peluru.

- **Bullet Damage Bonus:** Apabila peluru berhasil membunuh bot musuh, bot mendapatkan **poin sebesar 20% dari *damage*** yang dibuat kepada musuh yang terbunuh.
- **Survival Score:** Setiap ada bot yang mati, bot lainnya yang masih bertahan pada ronde tersebut mendapatkan **50 poin**.
- **Last Survival Bonus:** Bot terakhir yang bertahan pada suatu ronde akan mendapatkan **10 poin** dikali dengan banyaknya musuh.
- **Ram Damage:** Bot mendapatkan **poin sebesar 2 kalinya *damage*** yang dibuat kepada bot musuh dengan cara menabrak.
- **Ram Damage Bonus:** Apabila musuh terbunuh dengan cara ditabrak, bot mendapatkan **poin sebesar 30% dari *damage*** yang dibuat kepada musuh yang terbunuh.

Skor akhir bot adalah akumulasi dari 6 komponen diatas. Perlu diperhatikan bahwa game akan menampilkan berapa kali suatu bot meraih peringkat 1, 2, atau 3 pada setiap ronde. Namun, hal ini tidak dihitung sebagai komponen skor maupun untuk perangnya akhir. Bot yang dianggap menang pertempuran adalah bot dengan akumulasi skor tertinggi.

Untuk informasi lebih lengkap, silahkan buka dokumentasi Tank Royale pada link [berikut](#).

Starter Pack

Untuk tugas besar ini, game engine yang akan digunakan sudah dimodifikasi oleh asisten.

Berikut adalah beberapa perubahan yang dibuat oleh asisten dari game engine Tank Royale:

- **Theme GUI:** diubah menjadi light theme
- **Skor & Energi:** masing-masing bot ditampilkan di samping arena permainan agar lebih mudah diamati saat pertarungan
- **Turn Limit:** Durasi pertarungan tidak akan bergantung pada banyak ronde. Pada game engine ini, pertarungan akan berakhir apabila banyaknya turn sudah mencapai batas tertentu. Apabila batasan ini tercapai, ronde otomatis langsung berakhir dan pemenang pertarungan akan ditampilkan. Turn Limit dapat diatur pada menu “setup rules”. Source Code untuk game engine dan template bot telah disediakan pada tautan berikut.

[tubes1-if2211-starter-pack](#)

Adapun panduan mengenai cara menjalankan game engine, membuat bot, dan melihat referensi API dapat dilihat melalui tautan berikut: [Starter Guide](#)

LANDASAN TEORI

2.1 Dasar Teori Algoritma Greedy

2.1.1 Filosofi dan Konsep Dasar

Algoritma Greedy adalah paradigma desain algoritma yang berfokus pada pemecahan masalah optimasi melalui pendekatan heuristik langkah demi langkah (sekuensial). Prinsip fundamental dari algoritma ini yaitu greedy choice property yang punya makna membuat pilihan yang tampak paling optimal, menguntungkan, atau paling logis pada saat itu juga (lokal optimal), tujuan utamanya yaitu akumulasi dari keputusan-keputusan lokal tersebut akan berkonvergensi menjadi solusi yang optimal secara keseluruhan (global optimal).

Karakteristik komputasional utama dari algoritma Greedy adalah sifatnya yang myopic(rabun dekat) dan ireversibel. Algoritma ini tidak melakukan evaluasi terhadap konsekuensi masa depan dari sebuah keputusan, dan tidak memiliki mekanisme penelusuran balik (backtracking). Sekali suatu keputusan dieksekusi, state tersebut menjadi absolut dan tidak dapat dievaluasi ulang.

2.1.2 Elemen-Elemen Pembentuk Algoritma Greedy

Secara matematis dan arsitektural, algoritma Greedy dibangun oleh lima elemen inti yang saling berinteraksi. Dalam konteks agen otonom (seperti bot pada Robocode), elemen-elemen ini dievaluasi secara kontinu pada setiap siklus komputasi:

1. Himpunan Kandidat (C): Kumpulan seluruh keputusan, tindakan, atau aksi legal yang tersedia pada suatu state tertentu. Dalam permainan bot, ini mencakup himpunan diskrit perintah seperti berbelok, maju, mundur, merotasi radar, atau menembak dengan besaran daya tertentu.
2. Himpunan Solusi (S): Himpunan yang secara bertahap menyimpan kandidat terpilih. Pada akhir eksekusi, himpunan ini merepresentasikan sekuens komprehensif dari seluruh aksi yang telah dieksekusi agen dari awal hingga program berhenti.
3. Fungsi Seleksi (Selection Function): Komponen heuristik paling krusial yang memberikan nilai (bobot) pada setiap kandidat di dalam C, dan memilih kandidat dengan nilai terbaik. Evaluasi heuristik ini sering kali menggunakan perhitungan geometri spasial, seperti pencarian jarak terpendek (Euclidean distance):

$$d = \sqrt{(X_{target} - X_{bot})^2 + (Y_{target} - Y_{bot})^2}$$

4. Fungsi Kelayakan (Feasibility Function): Kondisi penjaga (guard clause) yang memvalidasi apakah kandidat yang dipilih oleh Fungsi Seleksi melanggar batasan (constraints) sistem. Misalnya, membatalkan aksi bergerak maju jika terdeteksi bahwa lintasan tersebut akan menabrak dinding arena.
5. Fungsi Objektif (Objective Function): Metrik akhir yang mengukur kualitas himpunan solusi. Fungsi ini bertujuan untuk melakukan maksimasi (seperti akumulasi skor, poin kerusakan, dan poin kelangsungan hidup) atau minimasi (seperti meminimalisir damage yang diterima).

2.1.3 Analisis Kompleksitas dan Limitasi

Kelebihan utama algoritma Greedy dalam sistem real-time itu efisiensi komputasinya. Karena keputusan diambil langsung berdasarkan state saat itu tanpa ekspansi pohon probabilitas pencarian, kompleksitas waktunya sangat rendah, umumnya berjalan pada $O(1)$ atau $O(N)$. Hal ini kritis sekali karena ketatnya batasan waktu komputasi.

Tapi, algoritma ini juga punya kelemahan inheren yaitu kerentanan terjebak dalam optimum lokal. Keputusan yang optimal pada waktu t bisa menjadi keputusan yang buruk pada waktu $t+1$ jika lingkungan sangat dinamis. Ketidadaan visi masa depan (predictive calculation) membuat algoritma murni Greedy rentan terhadap musuh yang menggunakan pola pergerakan acak atau adaptif.

2.2 Cara Kerja Program secara Umum

2.2.1 Paradigma Event-Driven pada Robocode Tank Royale

Robocode Tank Royale adalah permainan pemrograman di mana entitas bot virtual berkompetisi secara otonom. Program bot yang dikembangkan tidak memakai kontrol manual atau linear secara prosedural, tapi menggunakan arsitektur event-driven (berbasis kejadian).

Dalam arsitektur ini, program bertindak sebagai pendengar (listener) pasif yang bereaksi terhadap pemicu (trigger) spesifik dari server pertempuran, seperti OnScannedBot (saat radar menemukan musuh), OnHitWall (saat menabrak tembok), atau OnHitByBullet (saat terkena proyektil musuh). Pemrogram mendesain logika "otak" bot dengan menyuntikkan algoritma ke dalam fungsi-fungsi event handler tersebut.

2.2.2 Diskritisasi Waktu (*Rounds, Turns, dan Timeout*)

Lingkungan simulasinya sendiri itu berjalan dalam konsep waktu yang terdiskritisasi. Satu pertempuran dibagi menjadi beberapa round. Setiap round dipecah menjadi unit komputasi terkecil yang disebut turns atau ticks. Satu turn merepresentasikan satu putaran loop mesin permainan di mana semua pergerakan dan kalkulasi fisika diperbarui.

Ketegasan sistem ditunjukkan melalui mekanisme turn timeout. Server hanya memberikan jendela waktu komputasi yang sangat singkat (umumnya 30-50 milidetik per turn) untuk bot menyelesaikan kalkulasi algoritmanya. Kalau kompleksitas komputasi nya terlalu tinggi sehingga melebihi batas waktu ini, bot gagal mengirimkan perintah intent ke server, yang mengakibatkan bot melewatkan gilirannya secara fatal yang berakibat diam membeku di arena.

2.2.3 Anatomi dan Kinematika Agen

Sebuah agen bot direpresentasikan oleh tiga sub-komponen fisik yang dapat bergerak secara independen (asinkron) maupun terikat (sinkron):

- Body (Tubuh): Basis pergerakan motorik. Menentukan posisi koordinat (X, Y) bot. Kecepatan berbelok body bergantung secara invers pada kecepatan laju tangki saat itu (maksimal 10 derajat per turn dalam keadaan optimal).
- Gun (Meriam): Modul persenjataan. Kecepatan rotasi independen maksimal mencapai 20

derajat per turn.

- Radar (Pemindai): Modul sensor optik. Berfungsi mendeteksi musuh dalam radius linier maksimum 1200 piksel dengan kecepatan rotasi tertinggi (hingga 45 derajat per turn). Sebuah entitas hanya dianggap terdeteksi jika berada tepat di dalam sapuan sudut (scan arc) radar pada turn yang bersangkutan.

2.2.4 Fisika Permainan dan Manajemen Sumber Daya

Perancangan strategi algoritma harus memperhitungkan hukum fisika bawaan sistem, terutama terkait konversi energi dan termodinamika senjata:

1. Energi sebagai Daya Hidup: Setiap bot memulai dengan 100 poin energi. Energi akan berkurang bukan cuma dari damage yang diterima, tapi juga pas menembak meriam. Jika energi mencapai 0, agen menjadi nonaktif dan hancur jika menerima serangan sekecil apa pun. Sebaliknya, energi dapat diregenerasi jika peluru berhasil mengenai musuh (mendapatkan kembali 3 kali lipat dari energi proyektil yang dilontarkan).
2. Kalkulasi Balistik dan Gun Heat: Keputusan penembakan melibatkan trade-off linier antara firepower (daya energi yang dihabiskan) terhadap kecepatan peluru. Hubungan matematis ini direpresentasikan di mana peluru berdampak besar akan bergerak lebih lambat. Selain itu, setiap penembakan akan memanaskan senjata (Gun Heat). Algoritma tidak dapat mengeksekusi penembakan beruntun, algoritma harus melakukan pengecekan (Feasibility Function) hingga sistem pendingin menetralkan suhu meriam kembali ke angka nol.

APLIKASI STRATEGI GREEDY

3.1 Mapping Persoalan Robocode Menjadi Elemen Greedy

Berikut adalah hasil pemetaan komponen simulasi Robocode Tank Royale ke dalam elemen-elemen dasar algoritma Greedy pada arsitektur bot kami:

Elemen Greedy	Implementasi pada Bot Kami
Himpunan Kandidat (C)	Perintah aksi legal asinkron pada setiap tick (belok kanan/kiri, maju/mundur, rotasi radar, kalkulasi daya tembak 1.0 hingga 3.0).
Himpunan Solusi (S)	Rangkaian antrean aksi yang berhasil dieksekusi tanpa pembatalan (intent override) dari awal ronde sampai permainan berakhir.
Fungsi Seleksi	Memilih aksi pergerakan dan daya tembak berdasarkan heuristik lokal terbaik saat itu (misalnya: menembak dengan daya maksimum saat jarak target berada di bawah batas kritis)..
Fungsi Kelayakan	Validasi agar aksi tidak merugikan (contoh: mengecek <code>GunHeat == 0</code> sebelum menembak, dan kalkulasi jarak ke batas arena <code>WALL_MARGIN</code> untuk menghindari tabrakan dinding).
Fungsi Objektif	Memaksimalkan perolehan Bullet Damage sekaligus meminimalkan kerusakan fisik akibat benturan atau tembakan musuh.
Fungsi Solusi	Kondisi terminal permainan di mana bot kami bertahan sebagai penyintas terakhir (Last Survivor) atau hancur.

3.2 Eksplorasi 4 Alternatif Solusi Greedy

Kami merumuskan 4 alternatif bot dengan pendekatan fungsi heuristik Greedy yang saling bertolak belakang untuk diuji:

A. Alternatif 1 (GreedySniper):

Bot berfokus pada pertahanan pasif jarak jauh. Bot selalu memilih untuk mempertahankan proyektil tercepat dengan mengorbankan daya hancur.

Heuristik: (FirePower) dan menjaga jarak (Distance).

B. Alternatif 2 (GreedyRammer):

Bot berfokus pada agresi fisik dan daya hancur maksimal. Bot selalu memilih rute terpendek untuk menabrak target dan baru melepaskan tembakan di titik buta lawan.

Heuristik: jarak (Distance) dan mengeksekusi (FirePower).

C. Alternatif 3 (GreedyDodger):

Bot berfokus pada manuver evasif dan konservasi energi. Pergerakan selalu mengambil sudut ortogonal dari lawan, dan tembakan hanya dikalkulasi jika status energi bot surplus.

Heuristik: Sudut pergerakan 90 derajat secara absolut, syarat tembaknya adalah $Energy > 50$.

D. Alternatif 4 (MainBot):

Bot adaptif dengan penguncian target prediktif (*Predictive Targeting*). Pemilihan daya tembak bersifat dinamis dengan menyesuaikan jarak target saat itu, menggunakan *Adaptive Circle Strafing* yang berguna untuk mengoptimalkan pergerakan sambil mendeteksi dinding secara proaktif.

Heuristik: Penyesuaian daya tembak adaptif berdasarkan radius kedekatan dan prediksi lintasan tembak dengan probabilitas kena tertinggi.

3.3 Analisis Efisiensi dan Efektivitas

3.3.1 Efisiensi Komputasi

Semua logika heuristik pada keempat alternatif beroperasi dengan kompleksitas waktu $O(1)$ per pergantian milidetik (tick). Kalkulasi target bergantung sepenuhnya pada operasi trigonometri dasar (Math.Atan2, Math.Sin, Math.Cos). Implementasi Moving Average Filter pada MainBot menggunakan struktur data Queue berukuran statis (3 elemen), sehingga operasi penyisipan dan penghapusan data memori berkinerja konstan dan tidak membebani batas waktu eksekusi antrean mesin permainan.

3.3.2 Efektivitas Strategi

- Alternatif 1 (GreedySniper)

Kelebihan : Kecepatan peluru yang tinggi (karena daya tembak rendah 1.0) sangat sulit dihindari lawan dari kejauhan. Keamanan bot cukup terjamin di awal ronde.

Kekurangan : Menggunakan penargetan langsung (Direct Targeting) tanpa prediksi. Menghasilkan total kerusakan yang minim, terlebih lagi bot bergerak mundur secara buta jika lawan memaksa masuk ke radius dekat (< 300 piksel) yang membuatnya mudah tepojok.

- Alternatif 2 (GreedyRammer)

Kelebihan : Tingkat penyelesaian (kill rate) sangat tinggi saat melawan bot diam atau bot yang berjalan lambat. Logika kode sangat sederhana dan bebas galat.

Kekurangan : Menerima kerusakan fisik berlebihan akibat tabrakan yang disengaja. Selain itu, bot juga rentan hancur di awal permainan melawan bot yang menggunakan strategi pergerakan memutar.

- Alternatif 3 (GreedyDodger)

Kelebihan : Manuver pergerakan menyamping (ortogonal 90 derajat) membuat bot ini paling sulit ditembak oleh musuh yang menggunakan penargetan langsung.

Kekurangan : Gun tidak menggunakan predictive aiming, sehingga akurasi tembakan lebih rendah. Strafe cycle yang tetap (18 tick) dapat diprediksi oleh musuh yang adaptif.

- Alternatif 4 (MainBot)

Kelebihan : Integrasi Predictive Targeting menghasilkan akurasi tembakan jauh di atas ketiga bot alternatif. Logika proaktif Wall Avoidance memastikan Circle Strafing berjalan tanpa hambatan tabrakan dinding. Manajemen daya tembak sangat optimal terhadap penggunaan energi.

Kekurangan : Proses sinkronisasi asinkron antara komponen bodi, meriam, dan radar sangat bergantung pada tingkat akurasi perhitungan arah. Apabila penyesuaian sudut kompas pada `TurnRemaining` tidak dikalkulasi secara presisi, sistem berisiko tinggi mengalami konflik antrean (*intent deadlock*) yang dapat menggagalkan keselarasan pergerakan ketiga komponen tersebut.

3.4 Strategi Greedy yang Dipilih

Berdasarkan analisis kelebihan dan kompensasi kekurangan pada Subbab 3.3, kami memutuskan untuk mengimplementasikan **Alternatif 4 (MainBot)** sebagai agen final dalam Tugas Besar ini. MainBot memiliki struktur heuristik paling matang karena mampu beradaptasi terhadap perubahan lingkungan secara asinkron, memprediksi arah lawan menggunakan *smoothing data*, dan menjaga kelangsungan hidup tanpa mengorbankan keluaran kerusakan tembakan.

IMPLEMENTASI ALGORITMA GREEDY

4.1 Implementasi 4 Alternatif Solusi (Pseudocode)

Pada bagian ini menyajikan representasi logika berupa pseudocode dari empat buah bot sebagai alternatif solusi strategi Greedy. Setiap alternatif solusi beroperasi mengambil keputusan lokal terbaik secara instan berdasarkan informasi lingkungan yang ditangkap radar.

4.1.1 Alternatif Solusi 1: Algoritma *Adaptive Predictive* dan *Moving Average Filter* (MainBot)

Strategi ini merupakan agen utama (asinkron) yang memprioritaskan prediksi titik temu peluru di masa depan menggunakan filter rata-rata (3-tick), dikombinasikan dengan pergerakan Circle Strafin yang secara proaktif menghindari dinding arena.

Algoritma Solusi_Greedy_MainBot (Siklus Pemindaian & Asinkron)

Input: event_scanned, posisi_bot, batas_arena, antrean_prediksi

Output: set_movement, set_radar, set_gun, fire_action

Mulai

```
// 1. UPDATE STATE DAN RADAR LOCKING
lastScannedTick ← TurnNumber
jarak ← Hitung_Jarak_Ke(event_scanned.X, event_scanned.Y)
absolute_bearing ← Hitung_Sudut_Absolut(event_scanned.X,
event_scanned.Y)
SetTurnRadarRight(Hitung_Shortest_Bearing(RadarDirection,
absolute_bearing))

// 2. HEURISTIK DAYA TEMBAK (GREEDY DISTANCE)
JIKA jarak < 200 MAKA fire_power ← 3.0
LAIN JIKA jarak < 400 MAKA fire_power ← 2.0
LAIN fire_power ← 1.5
AKHIR JIKA

// 3. PREDIKSI LINEAR & MOVING AVERAGE FILTER
bullet_speed ← 20.0 - (3.0 * fire_power)
travel_time ← jarak / bullet_speed
pred_X ← event_scanned.X + Sin(event_scanned.Direction) *
event_scanned.Speed * travel_time
pred_Y ← event_scanned.Y + Cos(event_scanned.Direction) *
event_scanned.Speed * travel_time

Masukkan(pred_X) ke antrean_prediksi_X
Masukkan(pred_Y) ke antrean_prediksi_Y
JIKA Ukuran(antrean) > 3 MAKA Keluarkan_Data_Terlama()

gun_bearing ← Hitung_Sudut_Absolut_Ke(RataRata(antrean_prediksi_X),
RataRata(antrean_prediksi_Y))
SetTurnGunRight(Hitung_Shortest_Bearing(GunDirection, gun_bearing))
```

```

    JIKA (Absolut(kemiringan_meriam) <= 3.5) DAN (GunHeat == 0) MAKA
        SetFire(fire_power)
    AKHIR JIKA

    // 4. WALL AVOIDANCE (PROAKTIF)
    JIKA Jarak_Ke_Dinding_Terdekat() < WALL_MARGIN MAKA
        Arahkan_Bodi_Ke(Pusat_Arena)
        SetForward(60)
        Keluar Prosedur // Hentikan eksekusi pergerakan strafing di bawah
    AKHIR JIKA

    // 5. CIRCLE STRAFING
    strafe_angle ← absolute_bearing + (90 * strafeDirection)
    SetTurnRight(Hitung_Shortest_Bearing(Direction, strafe_angle))

    JIKA jarak > 250 MAKA SetForward(50)
    LAIN JIKA jarak < 150 MAKA SetBack(50)
    LAIN SetForward(30)
    AKHIR JIKA
Selesai

```

4.1.2 Alternatif Solusi 2: Algoritma *Defensive Sniper* (Greedy Sniper)

Strategi ini pasif dan menggunakan pemanggilan sinkron (*blocking*). Fokus utamanya adalah menjaga jarak sejauh mungkin dari target dan menggunakan daya tembak rendah agar kecepatan luncur peluru maksimal.

```

Algoritma Solusi_Greedy_Sniper
Input: event_scanned
Output: movement_action, gun_action

Mulai
    jarak ← Hitung_Jarak_Ke(event_scanned.X, event_scanned.Y)
    arah ← Hitung_Arah_Ke(event_scanned.X, event_scanned.Y)

    TurnGunRight(Hitung_Shortest_Bearing(GunDirection, arah))

    // Heuristik: Mundur jika musuh masuk zona ancaman
    JIKA jarak < 300 MAKA
        Back(100)
    AKHIR JIKA

    JIKA GunHeat == 0 MAKA
        Fire(1.0) // Daya kecil, peluru cepat
    AKHIR JIKA

    Rescan()
Selesai

```

4.1.3 Alternatif Solusi 3: Algoritma *Aggressive Rammer* (GreedyRammer)

Strategi sinkron ini berfokus pada kerusakan maksimal dari benturan fisik dan tembakan jarak dekat, mengabaikan segala bentuk pertahanan.

```
Algoritma Solusi_Greedy_Rammer

Input: event_scanned

Output: movement_action, gun_action

Mulai
    jarak ← Hitung_Jarak_Ke(event_scanned.X, event_scanned.Y)
    arah ← Hitung_Arah_Ke(event_scanned.X, event_scanned.Y)

    TurnRight(Hitung_Shortest_Bearing(Direction, arah))
    TurnGunRight(Hitung_Shortest_Bearing(GunDirection, arah))

    // Heuristik: Tabrak langsung ke koordinat musuh
    Forward(jarak)

    JIKA jarak < 100 DAN GunHeat == 0 MAKA
        Fire(3.0) // Daya hancur maksimal
    AKHIR JIKA

    Rescan()
Selesai
```

4.1.4 Alternatif Solusi 4: Algoritma *Evasive Dodger* (GreedyDodger)

Strategi sinkron yang berfokus pada kelangsungan hidup (*survivability*) dan konservasi energi. Pergerakan selalu ortogonal terhadap posisi musuh.

```
Algoritma Solusi_Greedy_Dodger
Input: event_scanned, Energy
Output: movement_action, gun_action

Mulai
    arah ← Hitung_Arah_Ke(event_scanned.X, event_scanned.Y)

    // Heuristik: Selalu menyamping (90 derajat dari arah musuh)
    TurnRight(Hitung_Shortest_Bearing(Direction, arah + 90))
    Forward(100)
```



```

    TurnGunRight(Hitung_Shortest_Bearing(GunDirection, arah))

    // Heuristik: Tembak hanya jika energi surplus
    JIKA Energy > 50 DAN GunHeat == 0 MAKA
        Fire(1.5)
    AKHIR JIKA

    Rescan()
Selesai

```

4.2 Struktur Data, Fungsi, dan Prosedur

Bot utama kami (MainBot) diimplementasikan menggunakan arsitektur asinkron (intent-based API) yang memungkinkan meriam, radar, dan sasis beroperasi paralel tanpa saling menunggu. Berikut adalah pembagian struktur datanya:

4.2.1. Struktur Kelas Data dan Atribut State

Enkapsulasi status internal bot dipelihara melalui properti instans kelas dengan pembagian berikut:

- `Queue<double> predXHistory & predYHistory`: Struktur data antrean (FIFO) yang digunakan sebagai memori jangka pendek menyimpan 3 kalkulasi prediksi titik temu koordinat proyektil terakhir. Berfungsi sebagai *Moving Average Filter* untuk meredam fluktuasi (*noise*) arah gerak musuh.
- `int strafeDirection`: Variabel skalar bernilai 1 atau -1 yang berfungsi sebagai pengali (pembulat arah). Variabel ini diinversi (dikalikan -1) setiap kali bot menabrak dinding atau objek fisik, sehingga arah manuver melingkar otomatis berbalik.
- `int lastScannedTick`: Variabel pencatat waktu absolut kapan radar terakhir kali menangkap musuh. Digunakan sebagai pemicu (trigger) untuk mereset putaran radar jika bot kehilangan jejak lebih dari 3 tick.
- `const double WALL_MARGIN`: Konstanta perimeter aman (80.0 piksel) yang mendefinisikan zona bahaya dinding arena.

4.2.2. Variabel Global (State Tracking dari API)

Properti bawaan dari mesin gim yang diakses secara real-time pada setiap daur simulasi:

- **X & Y**: Titik posisi koordinat kartesius bot saat ini, dievaluasi secara kontinu terhadap dimensi batas `ArenaWidth` dan `ArenaHeight`.
- **GunHeat**: Indikator suhu laras meriam. Menjadi gerbang logika utama (`GunHeat == 0`) yang wajib dipenuhi sebelum perintah `SetFire` dieksekusi.
- **TurnRemaining & DistanceRemaining**: Indikator status penyelesaian rotasi dan jarak tempuh. Digunakan untuk mencegah konflik antrean perintah (misalnya, tidak memberi perintah laju baru jika jarak target masih dieksekusi).

4.2.3. Fungsi dan Prosedur Utama (Method)

Berikut adalah fungsi-fungsi kritis yang mengendalikan alur algoritma asinkron bot kami:

- **public override void Run():** Prosedur gerbang utama (entry point). Mengatur flag independensi subsistem (AdjustGunForBodyTurn dan AdjustRadarForGunTurn), serta mengeksekusi antrean perintah secara simultan menggunakan perintah Go() dalam perulangan tak hingga.
- **public override void OnScannedBot(ScannedBotEvent e):** Prosedur utama tempat fungsi objektif *Greedy* dijalankan. Meliputi kalkulasi heuristik *Predictive Targeting* menggunakan Math.Atan2, evaluasi kontrol jarak dinamis, serta injeksi data koordinat ke dalam antrean *Moving Average*.
- **public override void OnHitBot(HitBotEvent e) dan OnHitWall(HitWallEvent e):** Prosedur interupsi fisik darurat. Keduanya memicu mekanisme pertahanan dengan mengubah nilai strafeDirection menjadi negatifnya, memaksa bot berbalik arah seketika dan mundur menjauhi objek tabrakan.
- **private bool IsNearWall():** Fungsi predikat boolean (proaktif) yang memeriksa apakah posisi absolut proyektil atau bot berada di dalam batas toleransi WALL_MARGIN.

4.3 Pengujian Pertandingan Free-for-All (FFA) 4 Bot

No	Kondisi Uji / Skenario	Hasil Skor	Catatan Kejadian Unik
1	Sesi 1: FFA Normal. Awal ronde didominasi agresi jarak dekat.	Menang (Peringkat 1) Skor: 363	GreedyMainBot berhasil selamat dari serbuan awal, mencetak nilai Survival maksimal (250). Terdapat banyak benturan fisik yang menghasilkan Ram Damage (44 poin).
2	Sesi 2: FFA Normal. GreedyRammer sangat agresif mengejar target.karena Bullet tidak dapat mengenai target.	Menang (Peringkat 1) Skor: 275	GreedyMainBot mampu menetralkan GreedyRammer yang finis di peringkat 2. Eksekusi tembakan jarak dekat (fire_power = 3.0) efektif menumbangkan penabrak.
3	Sesi 3: FFA Normal. GreedyMainBot sering tersudut ke area dinding.	Menang (Peringkat 1) Skor: 364	Terjadi lonjakan sangat ekstrem pada Ram Damage (116 poin). Saat Wall Avoidance aktif, MainBot secara tidak sengaja menabrak balik musuh yang menghalanginya.
4	Sesi 4: FFA Normal. GreedyDodger bertahan hingga akhir ronde bersama	Kalah (Peringkat 2) Skor: 332 (Kalah	Akurasi peluru MainBot (30 poin)

	MainBot.	dari Dodger: 355)	menurun drastis. Prediksi tembakan gagal mengunci GreedyDodger yang bermanuver menyamping secara konstan.
5	Sesi 5: FFA Normal. Posisi awal ronde tersebar merata di arena.	Menang Mutlak (Peringkat 1) Skor: 410	Kinerja paling optimal. Keseimbangan antara Bullet Damage (58 poin) dan Ram Damage (72 poin) membuktikan kelancaran asinkronisasi antara meriam dan bodi.

4.4 Analisis Hasil Pengujian

4.4.1 Keberhasilan Strategi Greedy

Dari lima skenario pengujian, GreedyMainBot berhasil mendominasi dan memenangkan empat pertandingan. Hasil ini secara empiris memvalidasi keunggulan strategi *Circle Strafing* yang dikombinasikan dengan *Predictive Targeting* dalam pertempuran berskala jamak (FFA).

Pada pengujian 1, 2, dan 5, strategi *Greedy* adaptif terbukti sukses besar. Heuristik yang mengatur jarak secara dinamis—maju saat musuh menjauh (> 250 piksel) dan mundur saat musuh terlalu dekat (< 150 piksel)—berhasil membuat MainBot selalu berada di jarak tembak ideal. Hal ini tercermin dari tingginya metrik *Survival Score* yang konsisten berada di angka 200 hingga 250, membuktikan bahwa MainBot sering menjadi penyintas terakhir (*Last Survivor*).

Pengujian 3 memperlihatkan temuan tak terduga yang menguntungkan. Ram Damage melonjak tajam hingga 116 poin. Hal ini membuktikan bahwa ketika fitur Wall Avoidance terpicu (memaksa tank berbelok menuju pusat arena), MainBot bertindak layaknya Rammer defensif yang menabrak dan melindas musuh yang mencoba memojokkannya di pinggir arena. Sinkronisasi perintah asinkron memastikan meriam tetap menembak (dengan daya 3.0) saat tabrakan terjadi.

4.4.2 Kondisi Strategi Greedy Tidak Optimal

Pada pengujian 4, GreedyMainBot gagal mempertahankan dominasinya dan takluk dari GreedyDodger pada fase akhir ronde. Kegagalan ini menyoroti keterbatasan fundamental dari heuristik *Linear Predictive Aiming* dan *Moving Average Filter* yang diimplementasikan.

Dalam duel melawan GreedyDodger yang memiliki perilaku pergerakan *strafing* ortogonal (tegak lurus 90 derajat secara absolut), filter prediksi 3-*tick* milik MainBot mengalami *lag* kalkulasi. Algoritma MainBot memproyeksikan lintasan musuh sebagai garis lurus yang konstan. Namun, setiap kali GreedyDodger memantul dari dinding atau sedikit mengubah arah orientasinya, koordinat aktualnya saat peluru tiba sudah bergeser secara signifikan dari prediksi.

Kondisi ini menggambarkan batas toleransi dari heuristik *Greedy* prediktif: keputusan untuk menembak didasarkan pada asumsi bahwa probabilitas lintasan masa depan bersifat konstan. Melawan bot dengan pola gerak acak atau *strafing* kaku seperti Dodger, tembakan MainBot membuang energi secara sia-sia.

Selain itu, karena MainBot menggunakan radar yang selalu memindai dan mereset target berdasarkan persinggungan terdekat, fokus tembakan sering terpecah di tengah keramaian. MainBot tidak memiliki mekanisme prioritas ancaman (misalnya: menargetkan musuh dengan sisa HP terendah), sehingga pada pengujian ke-4, energi bot habis terkuras sebelum berhasil melumpuhkan musuh yang paling lincah.

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan perancangan, implementasi, dan pengujian strategi Greedy pada simulasi Robocode Tank Royale, dapat ditarik beberapa kesimpulan bahwa pemilihan Alternatif Solusi Dari empat algoritma yang dieksplorasi (GreedySniper, GreedyRammer, GreedyDodger, dan MainBot), implementasi algoritma Adaptive Predictive Greedy pada MainBot terbukti paling efektif. Algoritma ini menyeimbangkan keluaran serangan (Bullet Damage dan Ram Damage) dengan tingkat kelangsungan hidup (Survival Score) yang optimal pada berbagai skenario pertempuran.

Penggunaan heuristik pergerakan berbasis jarak (kontrol maju/mundur pada radius 150–250 piksel) yang digabungkan dengan manuver Circle Strafing dan penghindaran dinding proaktif (Wall Avoidance) berhasil menghindarkan MainBot dari serangan lawan yang menggunakan penargetan langsung (Direct Targeting).

Algoritma Linear Predictive Targeting dengan Moving Average Filter 3-tick sangat akurat untuk target yang bergerak lurus. Namun, strategi ini terbukti kurang optimal dan mengalami keterlambatan kalkulasi (lag) saat melawan bot dengan pola pergerakan acak atau strafing kaku ortogonal (seperti GreedyDodger), yang berujung pada terbuangnya energi peluru secara sia-sia.

Penerapan API asinkron untuk memisahkan antrean instruksi pemindai, meriam, dan sasis pada MainBot berhasil mencegah konflik perintah (intent deadlock), memastikan bot beroperasi secara real-time dengan kompleksitas waktu $O(1)$ per pergantian tick.

5.2 Saran

Meningkatkan *Predictive Targeting* dari model linear menjadi *Circular Targeting* atau *Pattern Matching* agar meriam dapat memprediksi dan melacak musuh yang bergerak secara melingkar, acak, maupun non-linear.

Memasukkan fungsi heuristik tambahan yang mampu menyeleksi dan mengunci musuh berdasarkan ancaman tertinggi atau sisa *Health Points* (HP) terendah. Hal ini akan mencegah terpecahnya fokus serangan bot pada pertempuran berskala multikandidat (FFA).

Mengintegrasikan evaluasi cadangan energi internal bot sebelum mengeksekusi tembakan berdaya maksimum (3.0). Tembakan berdaya tinggi sebaiknya dibatasi hanya pada probabilitas kena (*hit rate*) yang absolut untuk menghindari lumpuhnya bot akibat kehabisan energi operasional.

LAMPIRAN

Tautan GitHub: <https://github.com/DanamiStartrail/Tubes-Stima.git>

<https://github.com/DanamiStartrail/Tubes-Stima/src/MainBot.git>

Video Demo:

No	Poin	Ya	Tidak
1	Bot dapat dijalankan pada Engine yang sudah dimodifikasi asisten.		
2	Membuat 4 solusi greedy dengan heuristic yang berbeda.		
3	Membuat laporan sesuai dengan spesifikasi.		
4	Membuat video bonus dan diunggah pada Youtube.		

DAFTAR PUSTAKA

- [1] Munir, Rinaldi. (2010). Diktat Kuliah Strategi Algoritma. Bandung: STEI ITB.
- [2] Dokumentasi Resmi Robocode Tank Royale API.